

BSD 314: Programming in Python

Semester Assignment – Technical Report

Student Performance Analytics and Management System

Field	Details
Student Name	Denis Macharia Mwai
Admission Number	BSCIT-01-0128/2025
Programme	Bachelor of Science in Information Technology
Course Code	BSD 314
Course Title	Programming in Python
Institution	Zetech University
Assignment Weight	10%
Total Marks	100
GitHub Repository	https://github.com/deikas123/BSD-314-Assignment-TWO.git

Table of Contents

1. Introduction
 2. Question One – Software Analysis and Design
 3. Question Two – Python Application Development
 4. Question Three – Object-Oriented Programming
 5. Question Four – Data Analytics and Visualization
 6. Question Five – Software Engineering Project
 7. Conclusion
 8. References
-

1. Introduction

Universities handle large volumes of student academic records every semester. Recording marks and computing grades manually is slow and can lead to mistakes. For this assignment I developed a **Student Performance Analytics and Management System** in Python. The application stores student details, records marks, calculates grades, analyses performance, and produces charts.

I used Python 3.13 together with functions, modules, lists, tuples, dictionaries, OOP, file handling, Pandas, NumPy and Matplotlib as covered in the BSD 314 course.

2. Question One: Software Analysis and Design (20 Marks)

2.1 Requirements Analysis (a)

Functional Requirements

Functional requirements describe what the system must do:

ID	Requirement	Description
FR1	Student Registration	Capture student ID, name, age, gender, course and year of study
FR2	Update / Delete Student	Allow modification or removal of student records
FR3	Search Student	Find a student by ID or name
FR4	Enter Marks	Record scores for different subjects
FR5	Calculate Average	Compute mean score for a student
FR6	Compute Grade	Assign letter grade (A–E) based on average
FR7	Display Transcript	Show a formatted academic transcript
FR8	Data Analytics	Clean data and compute statistical measures

FR9	Visualization	Generate bar, histogram, pie and line charts
-----	---------------	--

Non-Functional Requirements

Non-functional requirements describe quality attributes of the system:

ID	Requirement	Explanation
NFR1	Usability	Menu-driven interface that is easy for a lecturer/admin to use
NFR2	Reliability	Exception handling to prevent crashes from invalid input or missing files
NFR3	Maintainability	Modular code structure with separate files for models, managers and analytics
NFR4	Portability	Runs on any OS with Python 3.11+ installed
NFR5	Performance	Suitable for small-to-medium class sizes (tens to hundreds of students)
NFR6	Data Integrity	Validation of scores (0–100) and prevention of duplicate student IDs

2.2 System Architecture (b)

Use Case Diagram

The use case diagram (see `diagrams/use_case_diagram.png`) shows the Administrator/Lecturer interacting with the following use cases:

- Register Student
- Update / Delete Student
- Enter Marks
- Compute Grades
- View Transcript

- Generate Analytics & Charts

System Architecture Diagram

The system follows a layered architecture (see `diagrams/architecture_diagram.png`):

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ User Interface Layer ■ ← main.py  
(console menus) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Application Logic  
Layer ■ ← StudentManager, AcademicManager  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Data Processing Layer ■ ←  
models, analytics, visualization ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■  
Data Storage Layer ■ ← CSV / JSON files  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

This separation makes the system easier to maintain. For example, the storage layer can later be replaced with a database without rewriting the menu logic.

2.3 Algorithm Design (c)

Pseudocode – Student Registration

```

ALGORITHM RegisterStudent BEGIN INPUT student_id, name, age, gender, course, year IF
student_id already exists THEN DISPLAY "Error: Duplicate ID" RETURN END IF IF age <= 0
OR name is empty THEN DISPLAY "Error: Invalid data" RETURN END IF CREATE new Student
object ADD student to students list SAVE list to JSON and CSV files DISPLAY
"Registration successful" END

```

Pseudocode – Marks Entry

```

ALGORITHM EnterMarks BEGIN INPUT student_id, subject, score SEARCH student by
student_id IF student not found THEN DISPLAY "Student not found" RETURN END IF IF
score < 0 OR score > 100 THEN DISPLAY "Invalid score" RETURN END IF STORE score in
student's marks dictionary SAVE updated data to files DISPLAY "Mark recorded" END

```

Pseudocode – Grade Computation

```

ALGORITHM ComputeGrade BEGIN INPUT student_id GET student marks IF no marks THEN
DISPLAY "No marks recorded" RETURN END IF average  $\leftarrow$  SUM(marks) / COUNT(marks) IF
average  $\geq$  70 THEN grade  $\leftarrow$  "A" ELSE IF average  $\geq$  60 THEN grade  $\leftarrow$  "B" ELSE IF
average  $\geq$  50 THEN grade  $\leftarrow$  "C" ELSE IF average  $\geq$  40 THEN grade  $\leftarrow$  "D" ELSE grade  $\leftarrow$ 
"E" END IF DISPLAY average and grade END

```

Flowcharts for registration and marks/grade computation are provided in:

- diagrams/flowchart_registration.png
- diagrams/flowchart_marks_grade.png

2.4 Critical Evaluation (d)

Justification of Software Design

The layered modular design was chosen because:

1. **Separation of concerns** – UI, logic and storage are independent.
2. **Reusability** – classes like `Student` and `PerformanceAnalyzer` can be reused in other modules.
3. **Easier testing** – each module can be tested separately.
4. **Scalability** – new features (e.g. web UI) can sit on top of existing logic.

Why Python is Appropriate

Python is suitable for this project because:

- It has a clear, readable syntax which reduces development time.
 - Built-in data structures (lists, dictionaries, tuples) fit student/marks data well.
 - Libraries such as Pandas, NumPy and Matplotlib support analytics and visualization without extra low-level coding.
 - Strong OOP support allows modelling of Person, Student and Course.
 - File handling for CSV/JSON is simple.
 - Large community and documentation make troubleshooting easier for students.
-

3. Question Two: Python Application Development (25 Marks)

3.1 Application Overview

The application is menu-driven and launched using:

```
python main.py
```

Main Menu options:

1. Student Management
2. Academic Management
3. Reports & Analytics
4. Data Visualization
5. OOP Example

6. Exit

3.2 Programming Concepts Demonstrated

Concept	Where Used
Functions	All menu handlers and utility functions
Modules	<code>models</code> , <code>student_manager</code> , <code>academic_manager</code> , <code>file_handler</code> , <code>analytics</code> , <code>visualization</code>
Lists	<code>students</code> list in <code>StudentManager</code> and <code>PerformanceAnalyzer</code>
Tuples	<code>SUBJECTS</code> and <code>GRADE_SCALE</code> (immutable collections)
Dictionaries	Student marks (<code>subject</code> → <code>score</code>), grade distribution
Loops	Searching students, displaying tables, seeding data
Conditionals	Grade bands, menu choices, validation checks

3.3 Student Management Features

- **Register Student** – validates ID uniqueness and age
- **Update Student** – modify name, age, gender, course or year
- **Delete Student** – removes record after confirmation
- **Search Student** – by ID (exact) or name (partial match)
- **Display Students** – tabular listing of all registered students

3.4 Academic Management Features

- **Enter Marks** – subject selection from a fixed tuple or custom subject
- **Calculate Average** – mean of all recorded subject scores
- **Compute Grade** – letter grade using university-style bands
- **Display Transcript** – formatted transcript with per-subject grades

Data is persisted automatically to `data/students.json`, `data/students.csv` and `data/marks.csv`.

4. Question Three: Object-Oriented Programming (20 Marks)

4.1 Class Design

Class	Role
Person	Base class with name, age, gender
Student	Inherits Person; adds ID, course, year, marks
Course	Represents a taught unit (code, name, credits)
PerformanceAnalyzer	Analyses a collection of Student objects

UML Class Diagram: `diagrams/uml_class_diagram.png`

4.2 OOP Principles Demonstrated

Encapsulation

Attributes are stored with underscore prefix (`_name`, `_marks`). Access is controlled through getters and setters such as `get_name()` and `set_age()`, which also validate input.

Inheritance

Student inherits from Person using `super().__init__()`. Student reuses person attributes and adds academic fields.

Polymorphism

Both Person and Student implement `display_info()`. When called on different objects in a list, the correct version runs based on the object type.

Constructors

Each class has an `__init__` method that initialises object state when created.

Method Overriding

`Student.display_info()` overrides `Person.display_info()` to include student ID, course, average and grade.

4.3 Advantages of OOP over Procedural Programming

Aspect	Procedural	OOP
Structure	Functions + data separate	Data and behaviour grouped in classes
Reuse	Copy-paste or shared functions	Inheritance and composition
Maintenance	Harder as program grows	Changes localised to classes
Modelling	Less natural for real-world entities	Student, Course map clearly to objects
Extensibility	Often requires rewriting	New subclasses can extend behaviour

For this system, OOP made it natural to represent students as objects that “know” how to calculate their own average and grade.

5. Question Four: Data Analytics and Visualization (20 Marks)

5.1 Data Cleaning (a)

Using Pandas, the system:

1. **Detects missing values** – `df.isnull().sum()`
2. **Removes duplicates** – based on `(student_id, subject)`
3. **Validates records** – scores coerced to numeric; values outside 0–100 removed; empty subjects dropped

A cleaning report is printed showing original rows, missing counts, duplicates removed and final row count. Cleaned data is saved to `data/marks_cleaned.csv`.

5.2 Statistical Analysis (b)

Using NumPy and Pandas the following are computed:

- Mean
- Median
- Mode
- Standard Deviation

- Maximum and Minimum scores

Per-subject and per-student averages are also generated to support deeper analysis.

5.3 Data Visualization (c)

Charts are generated with Matplotlib and saved under `outputs/charts/`:

Chart	File	Interpretation
Bar Chart	<code>bar_chart.png</code>	Compares average performance across subjects
Histogram	<code>histogram.png</code>	Shows how scores are distributed; mean line marked
Pie Chart	<code>pie_chart.png</code>	Shows proportion of students in each grade band
Line Graph	<code>line_graph.png</code>	Ranks students by average score

5.4 Evaluation and Recommendations (d)

From sample data analysis:

- Class performance varies widely (high standard deviation indicates mixed ability).
- Top performers (e.g. students with averages above 80) can be challenged with enrichment tasks.
- Students below 50% (fail/borderline) need academic intervention.

Recommended interventions:

1. Remedial / tutorial classes for weak subjects (especially where subject averages are lowest).
2. Peer tutoring pairing high and low performers.
3. Continuous assessment rather than relying only on end-of-term exams.
4. Academic counselling for students with consistently low scores.
5. Review of teaching methods where a whole subject underperforms.

6. Question Five: Software Engineering Project (15 Marks)

6.1 GitHub Repository Structure

```
BSD-314-Assignment-TW0/ ■■■ main.py ■■■ run_demo.py ■■■ requirements.txt ■■■  
README.md ■■■ student_system/ ■ ■■■ __init__.py ■ ■■■ models.py ■ ■■■  
student_manager.py ■ ■■■ academic_manager.py ■ ■■■ file_handler.py ■ ■■■  
analytics.py ■ ■■■ visualization.py ■■■ data/ ■■■ diagrams/ ■■■ outputs/charts/  
■■■ screenshots/ ■■■ report/
```

Suggested GitHub actions after creating a repository:

```
git init git add . git commit -m "Initial commit: Student Performance Management  
System" git branch -M main git remote add origin  
https://github.com/deikas123/BSD-314-Assignment-TW0.git git push -u origin main
```

6.2 Virtual Environment Configuration

```
python -m venv venv # Windows venv\Scripts\activate # Linux/Mac source  
venv/bin/activate pip install -r requirements.txt
```

Dependencies:

- pandas $\geq$ 2.0
- numpy $\geq$ 1.24
- matplotlib $\geq$ 3.7

6.3 Exception Handling Strategy

Scenario	Handling
Missing data files	Return empty list / DataFrame; create directory if needed
Invalid JSON	Catch <code>JSONDecodeError</code> , return empty list
Invalid age/score	Raise / catch <code>ValueError</code> with clear message
Duplicate student ID	Reject registration with error message
File I/O errors	Catch <code>IOError/OSError</code> and display message
Non-numeric input	Convert carefully; show user-friendly errors

6.4 Testing Strategy

Test Type	Approach
Manual functional testing	Walk through each menu option
Sample data demo	<code>python run_demo.py</code> seeds 8 students and verifies outputs
Boundary testing	Scores 0, 100, -1, 101; empty name; duplicate ID
File testing	Delete JSON and confirm system recreates storage
OOP demo	Menu option 5 verifies inheritance and overriding

6.5 Future Improvements

Enhancement	Benefit
Flask / Django web version	Browser-based UI for lecturers and students
MySQL / PostgreSQL	Better concurrency and large-scale storage
Authentication system	Secure login roles (admin, lecturer, student)
Machine Learning module	Predict at-risk students from historical marks
Cloud deployment	Access from anywhere (e.g. Render, Railway, AWS)
PDF report export	Automatic printable transcripts and analytics reports
GUI (Tkinter / PyQt)	Richer desktop experience without a browser

6.6 Strengths and Limitations

Strengths

- Complete end-to-end workflow from registration to visualization
- Clear modular and OOP design
- Persistent storage using both JSON and CSV
- Practical analytics for academic decision-making

- Documented diagrams and report

Limitations

- Console UI only (no GUI/web)
 - File-based storage is not ideal for many concurrent users
 - No user authentication
 - Limited to single-institution sample grading scheme
 - Charts require Matplotlib display/file save (not interactive dashboards)
-

7. Conclusion

In this assignment I was able to build a working Student Performance Analytics and Management System in Python. The project covered requirements analysis, algorithm design, modular programming, OOP, file handling, data analytics and visualization. Python worked well for this task because the syntax is easy to follow and libraries like Pandas and Matplotlib made the analysis and charting straightforward. In future I would like to improve the system by adding a web interface and connecting it to a database.

8. References

1. Python Software Foundation. *Python Documentation*. <https://docs.python.org/3/>
 2. Pandas Development Team. *Pandas Documentation*. <https://pandas.pydata.org/docs/>
 3. Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*.
 4. Hunter, J.D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*.
 5. Zetech University. BSD 314 Course Materials – Programming in Python.
-

Appendix A: How to Run (Quick Guide)

```
cd "BSD 314 Assignment TWO" python -m venv venv venv\Scripts\activate pip install -r requirements.txt python run_demo.py # load sample data and run analysis python main.py # interactive system
```

Appendix B: Grade Scale Used

Average	Grade
70 – 100	A
60 – 69	B
50 – 59	C
40 – 49	D
Below 40	E

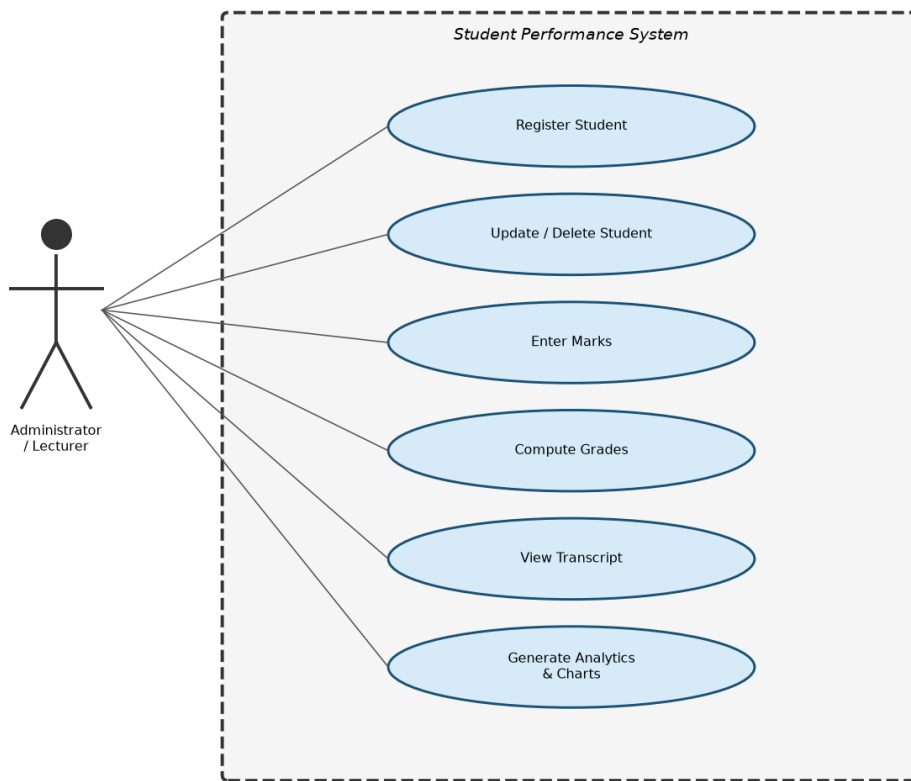
Appendix C: Sample Pseudocode Summary

See Section 2.3 and flowcharts in the `diagrams/` folder.

Appendix D: Design Diagrams

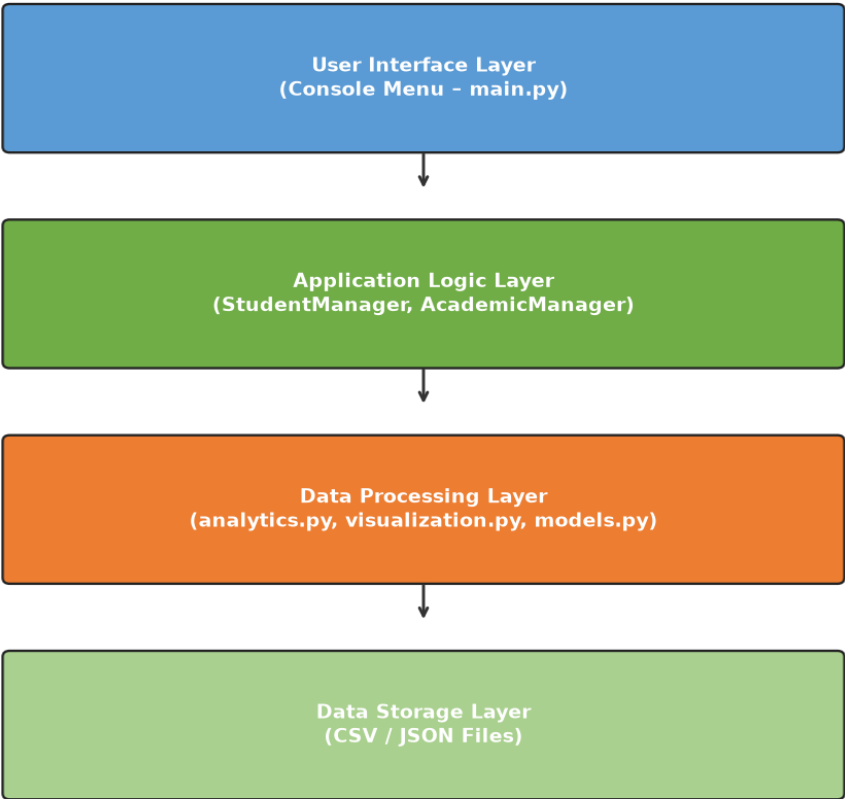
Use Case Diagram

Use Case Diagram - Student Performance Management System



System Architecture

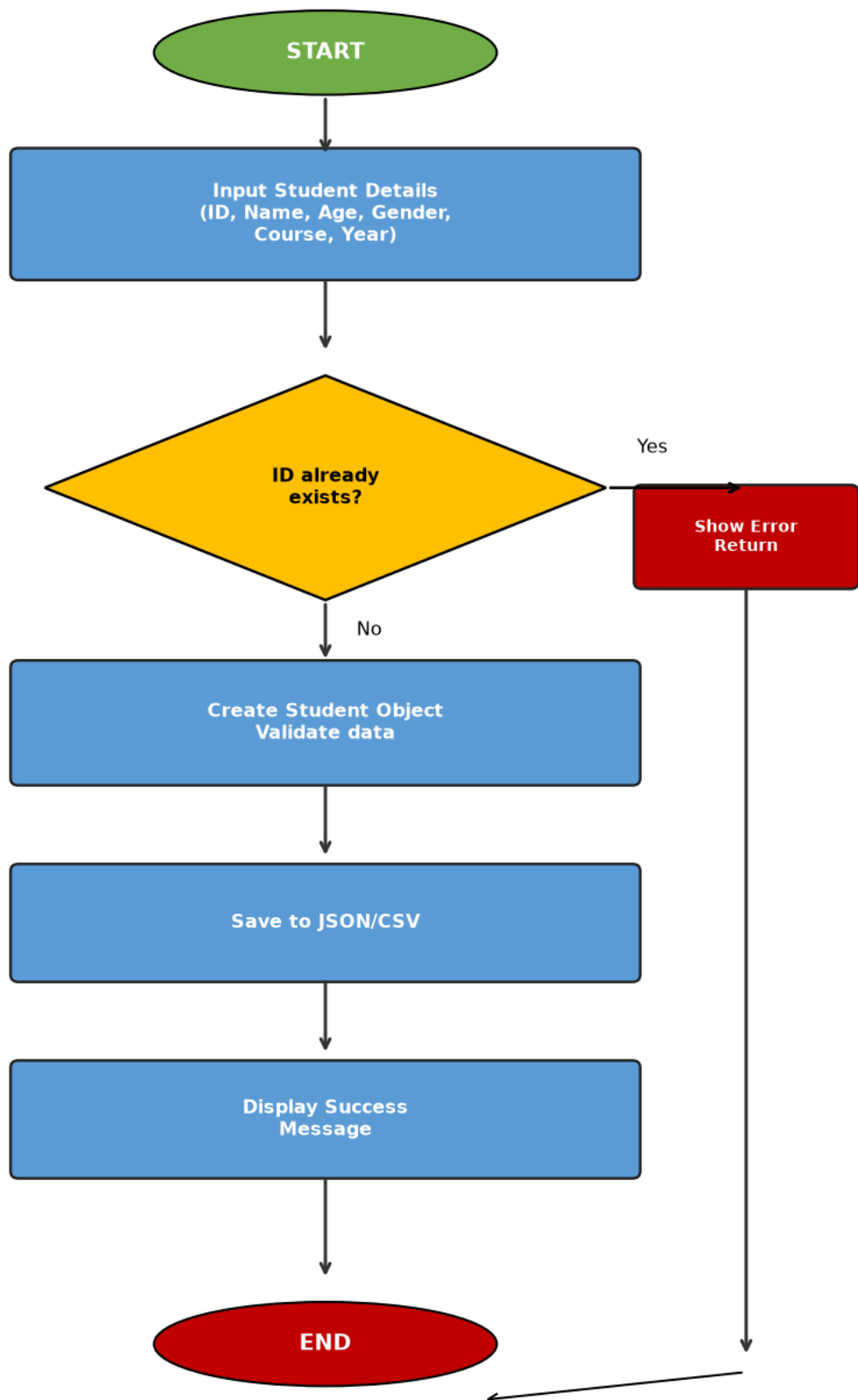
System Architecture Diagram



Data flows downward for writes; upward for reads/reports

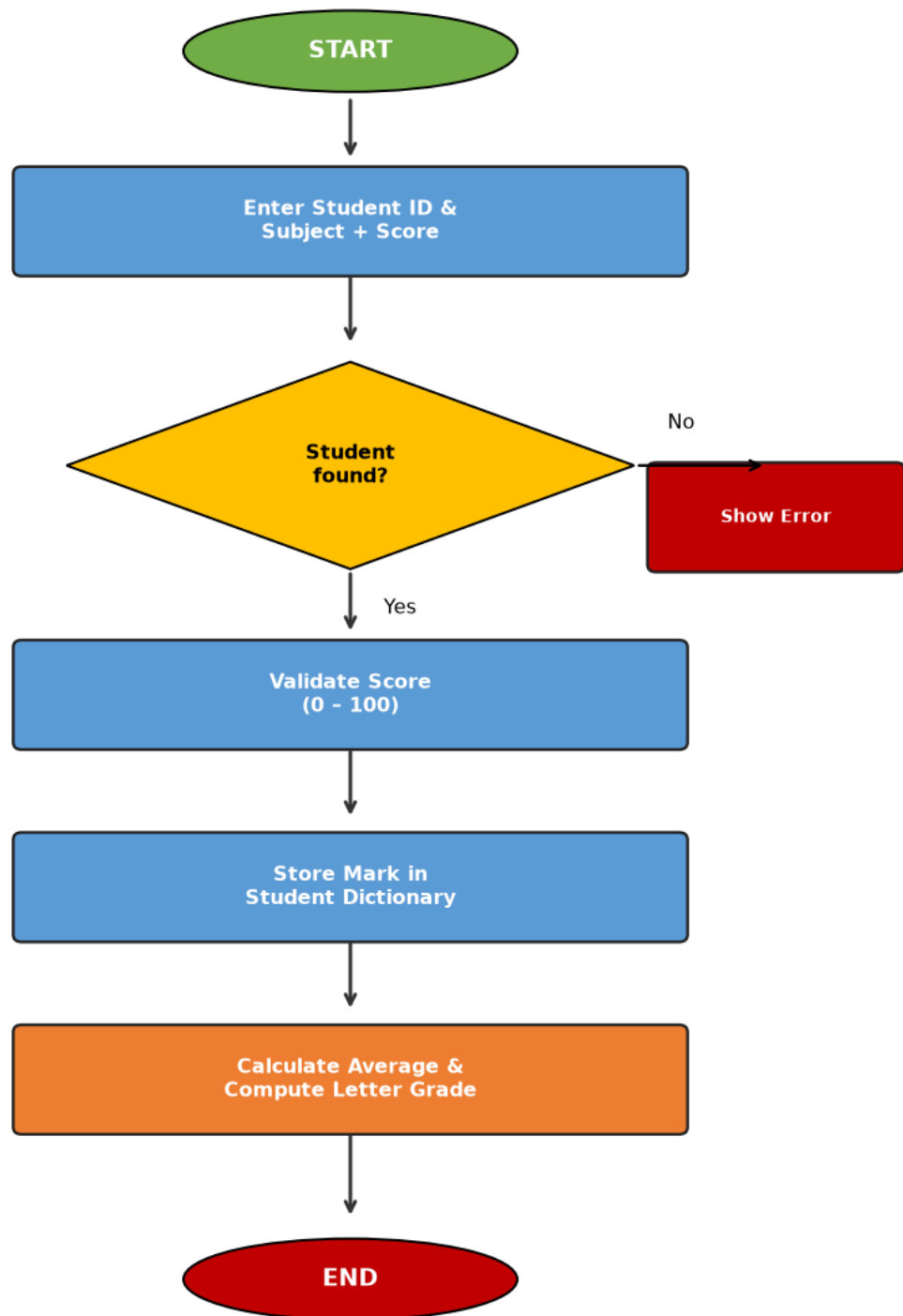
Flowchart – Student Registration

Flowchart: Student Registration



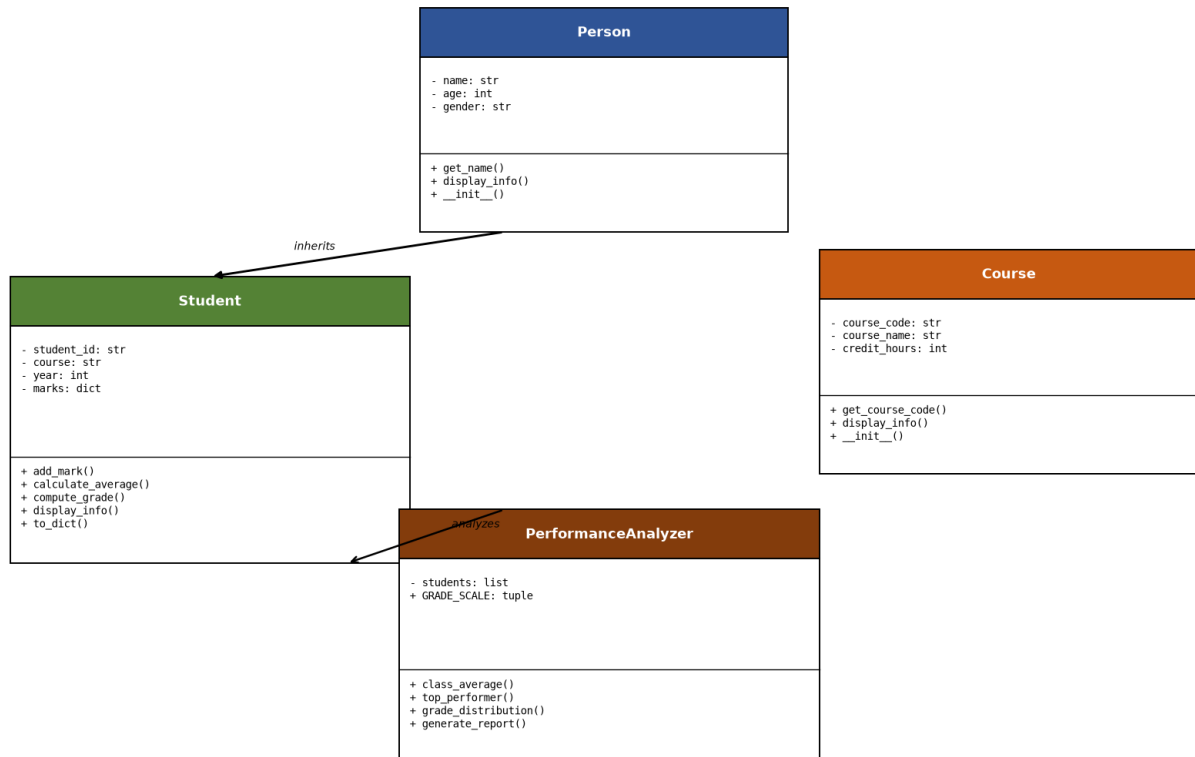
Flowchart – Marks Entry and Grade Computation

Flowchart: Marks Entry & Grade Computation



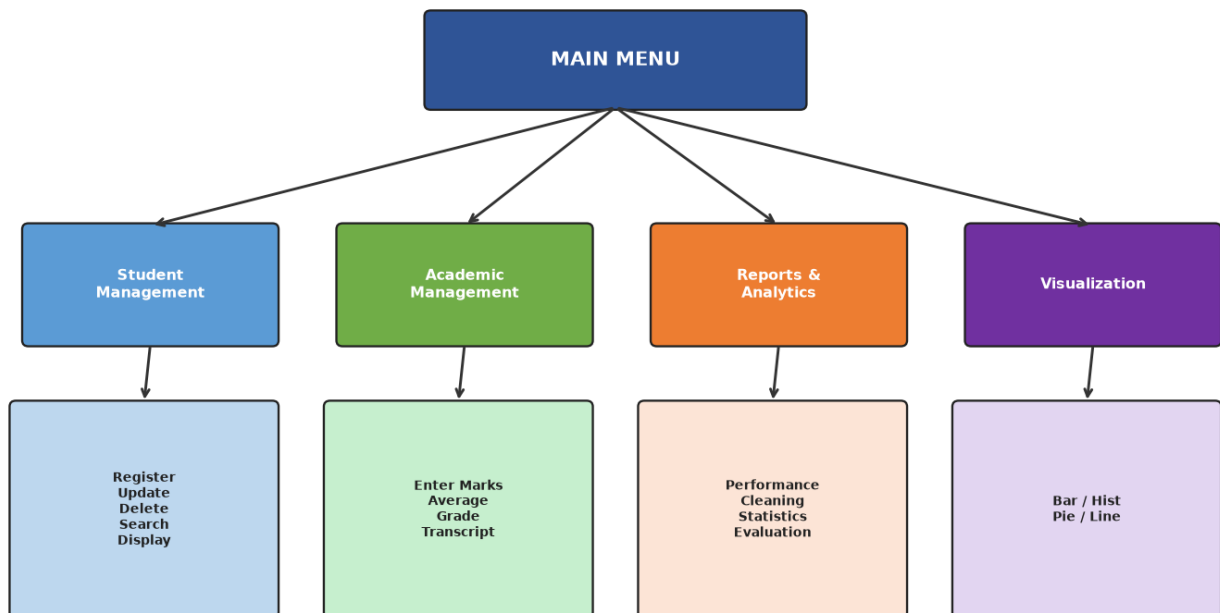
UML Class Diagram

UML Class Diagram



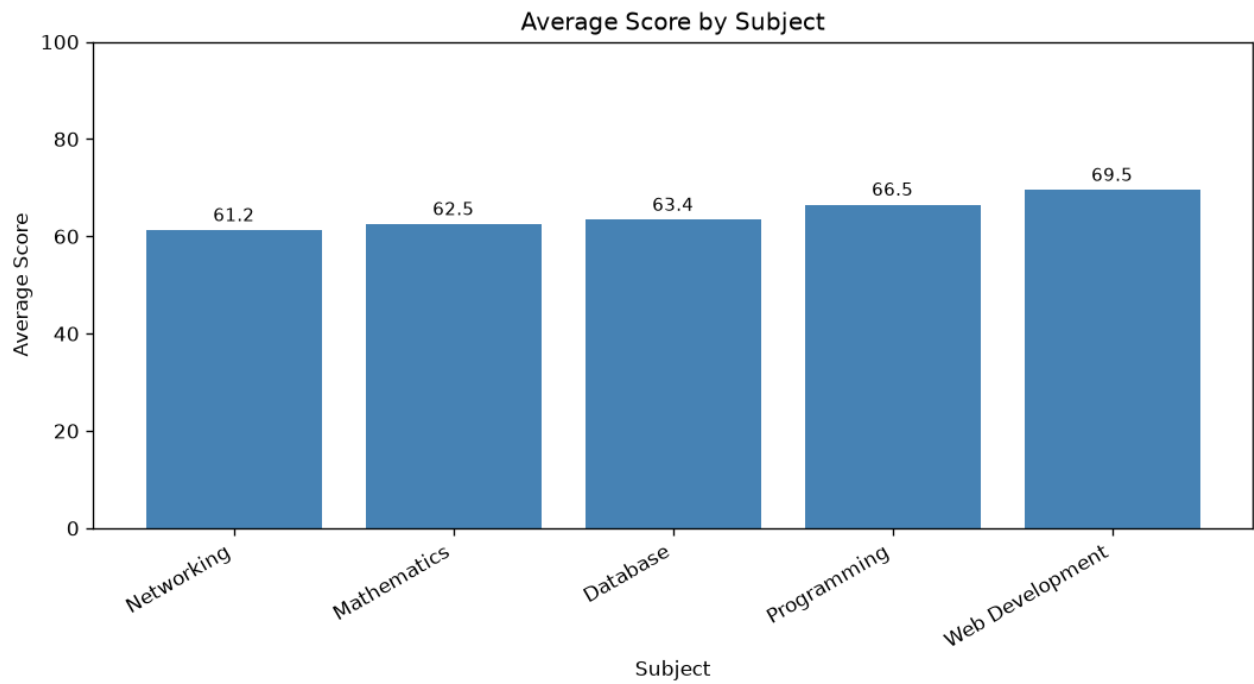
Menu Structure

Menu Structure

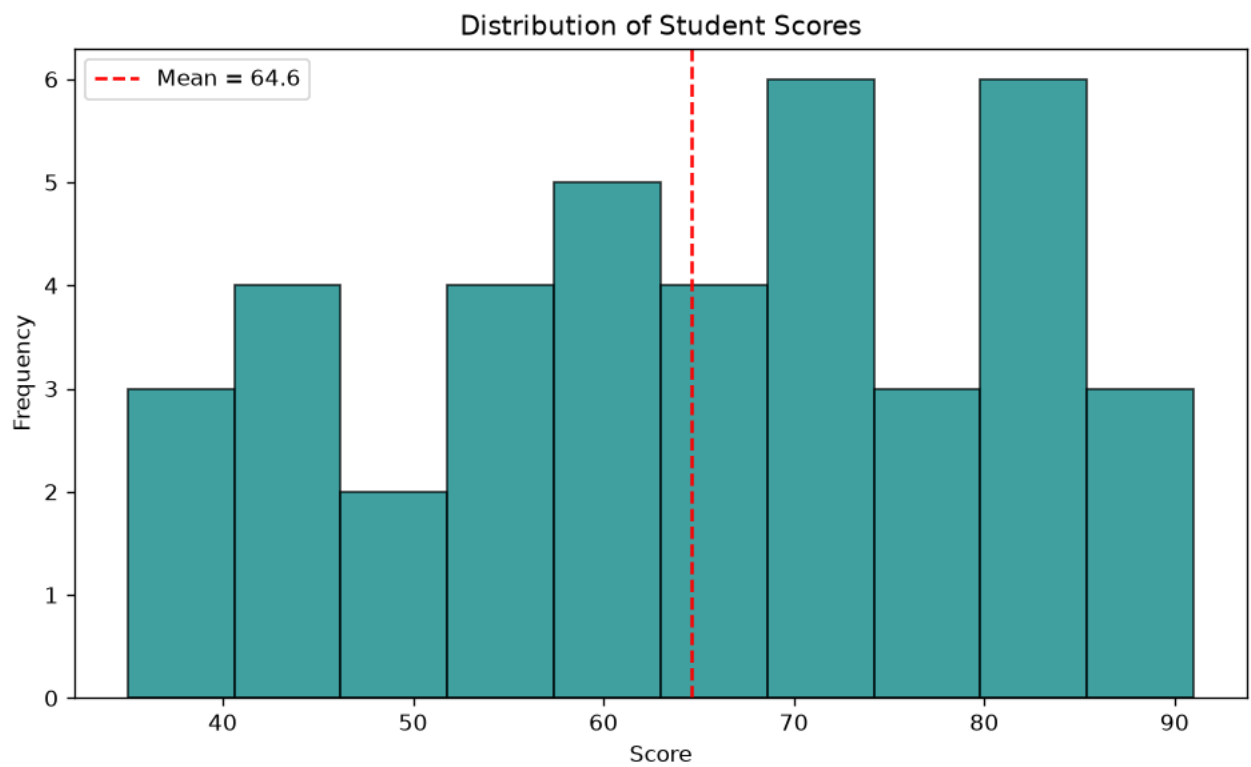


Appendix E: Visualizations

Bar Chart – Average by Subject

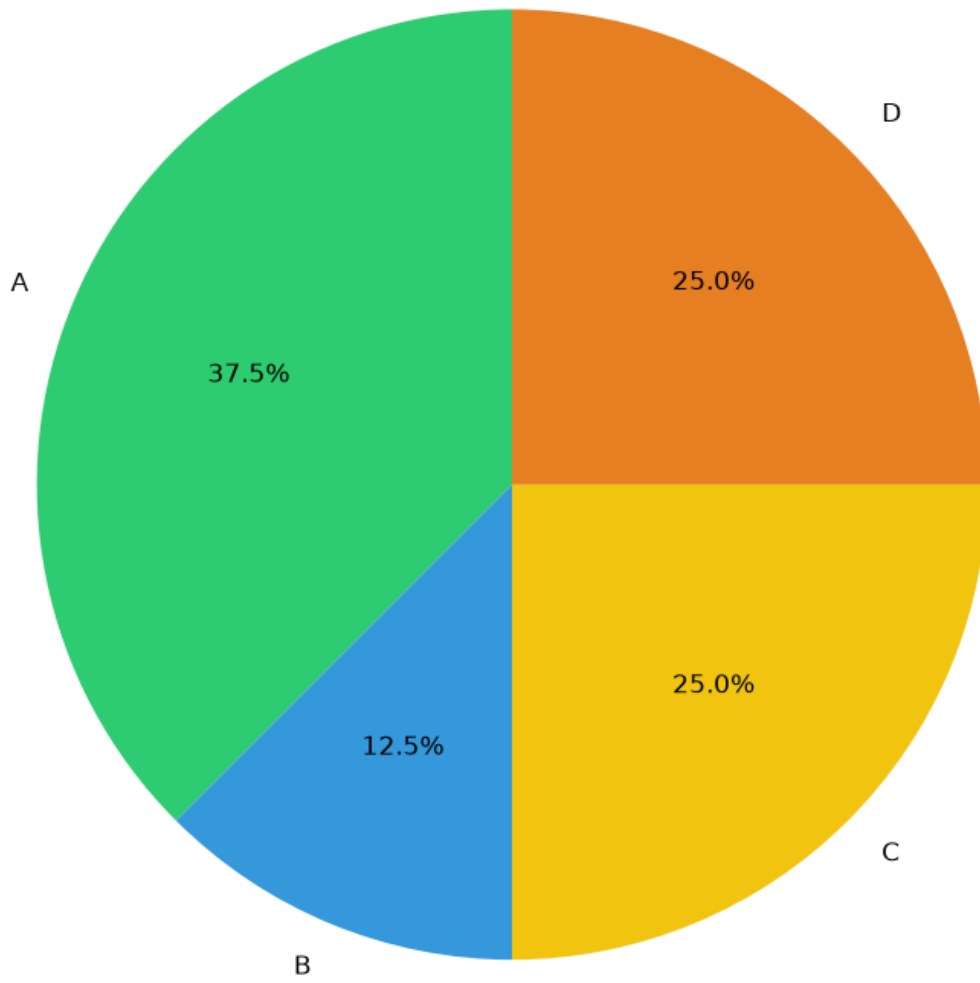


Histogram – Score Distribution

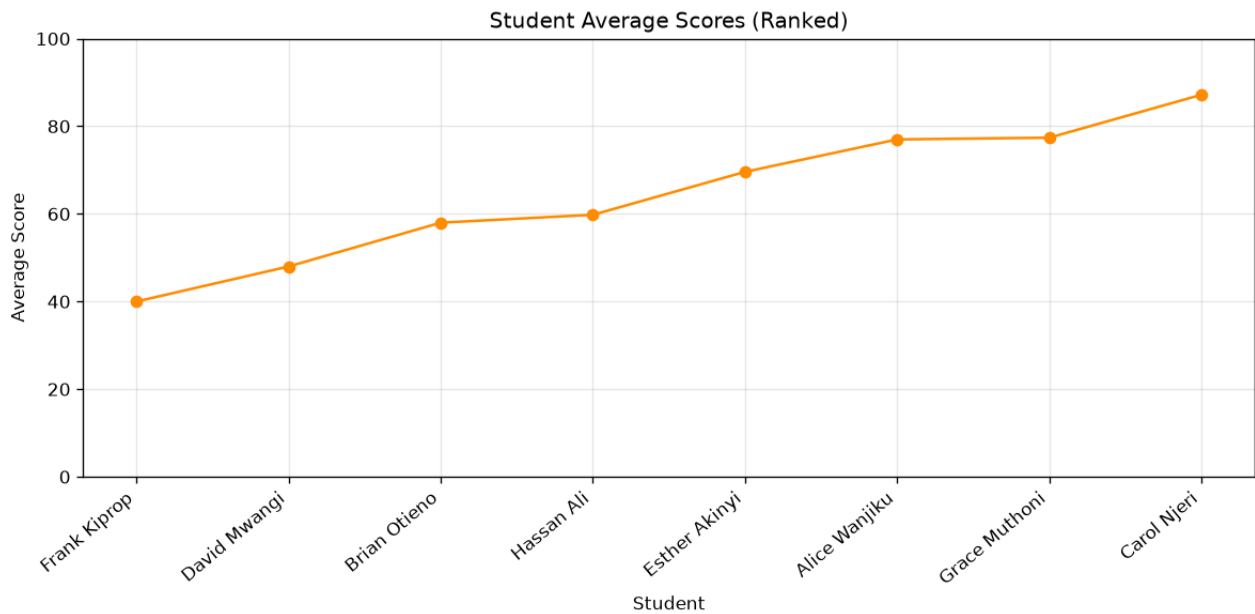


Pie Chart – Grade Distribution

Grade Distribution



Line Graph – Student Averages



Appendix F: Execution Screenshots

Main Menu

```
python main.py - Main Menu
```

```
=====
STUDENT PERFORMANCE MANAGEMENT SYSTEM
=====
1. Student Management
2. Academic Management
3. Reports & Analytics
4. Data Visualization
5. Demo OOP Features
0. Exit
-----
Enter choice: _
```

Display Students

Student Management – Display All Students

ID	Name	Age	Gender	Course	Year
STU001	Alice Wanjiku	20	F	BSc IT	2
STU002	Brian Otieno	22	M	BSc IT	2
STU003	Carol Njeri	19	F	BSc CS	1
STU004	David Mwangi	21	M	BSc IT	3
STU005	Esther Akinyi	20	F	BSc CS	2
STU006	Frank Kiprop	23	M	BSc IT	3
STU007	Grace Muthoni	21	F	BSc CS	2
STU008	Hassan Ali	22	M	BSc IT	1

Total students: 8

Academic Transcript

Academic Management – Transcript (STU001)

```
=====
          ACADEMIC TRANSCRIPT
=====
Student ID : STU001
Name       : Alice Wanjiku
Course     : BSc IT
Year       : 2
=====
Subject          Score    Grade
=====
Mathematics      72.0     A
Programming      85.0     A
Database         78.0     A
Networking       70.0     A
Web Development  80.0     A
=====
Average         : 77.0
Overall Grade: A
=====
```

Statistical Analysis

Reports & Analytics – Statistical Analysis

```
--- DATA CLEANING REPORT ---
Original rows      : 40
Missing values    : {'student_id': 0, 'name': 0, 'subject': 0, 'score': 0}
Duplicates removed : 0
Invalid removed   : 0
Final clean rows   : 40

--- STATISTICAL ANALYSIS ---
Number of scores  : 40
Mean              : 64.62
Median            : 65.0
Mode              : 55.0
Std Deviation     : 15.5
Maximum Score     : 91.0
Minimum Score     : 35.0

Class Average: 64.62 | Top Performer: Carol Njeri (87.2)
```

OOP Example

Demo OOP Features

1. INHERITANCE & CONSTRUCTORS
Person -> Student
Person: Name: Jane Doe, Age: 20, Gender: F
Student: John Kamau (STU001)
2. ENCAPSULATION (getters/setters)
Updated name via setter: John K. Kamau
3. METHOD OVERRIDING (display_info)
Student.display_info() includes ID, course, average, grade
4. POLYMORPHISM – same method, different behaviour per class
5. COURSE CLASS – BSD314 Programming in Python (3 credits)
6. PERFORMANCE ANALYZER – class average & grade distribution